VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018

PROJECT REPORT

On

DESIGN AND DEVELOPMENT OF PATIENT MONITORING AND IDENTIFICATION SYSTEM

Submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

ELECTRONICS AND INSTRUMENTATION ENGINEERING

Submitted by

ADARSH RAJ	**1BI22EI001**
KHUSHI HS	**1BI22EI021**
MUSKAAN HUSAIN	**1BI22EI029**

Under the guidance of

Prof NEETHASHREE NR.

Assistant Professor

Department of Electronics and Instrumentation Engineering

Carried out at

BANGALORE INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

K. R. Road, V. V. Pura, Bengaluru - 560 004

2025 - 2026

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018

PROJECT REPORT

On

DESIGN AND DEVELOPMENT OF PATIENT MONITORING AND IDENTIFICATION SYSTEM

Submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

ELECTRONICS AND INSTRUMENTATION ENGINEERING

Submitted by

ADARSH RAJ	**1BI22EI001**
KHUSHI HS	**1BI22EI021**
MUSKAAN HUSAIN	**1BI22EI029**

Under the guidance of

Prof. NEETHASHREE NR

Assistant Professor

Department of Electronics and Instrumentation Engineering

Carried out at

BANGALORE INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

K. R. Road, V. V. Pura, Bengaluru - 560 004

2025 - 2026

BANGALORE INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi)

K.R. Road, V. V. Pura, Bengaluru-560 004

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

CERTIFICATE

This is to certify that Project work entitled "**DESIGN AND DEVELOPMENT OF PATIENT MONITORING AND IDENTIFICATION SYSTEM**", the bonafide work carried out by **Mr. ADARSH RAJ (1BI22EI001), Ms. KHUSHI HS(1BI22EI021), Ms. MUSKAAN HUSAIN (1BI22EI0229),** students of Bangalore Institute of Technology in partial fulfilment for the award of Bachelor of Engineering in Electronics & Instrumentation Engineering under Visvesvaraya Technological University, Belagavi during the academic year 2025-2026. The report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Signature of the Guide **Prof. NEETHASHREE NR** Assistant Professor, Department of EIE, BIT	Signature of the HOD **Dr. ASWATHAPPA P.** Associate Professor & Head, Department of EIE, BIT	Signature of the Principal **Dr. SHANTAL S.** Principal, BIT, Bengaluru

EXTERNAL VIVA

Name of Examiner	Signature with Date
1._____	1._____
2._____	2._____

BANGALORE INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi)

K.R. Road, V. V. Pura, Bengaluru-560 004

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

DECLARATION

We, **ADARSH RAJ, KHUSHI HS,** and **MUSKAAN HUSAIN**, bearing USN **1BI22EI001**, **1BI22EI021**, and **1BI22EI029**, students of 7th Semester B.E in Electronics & Instrumentation Engineering, Bangalore Institute of Technology declare that the Project work entitled "**DESIGN AND DEVELOPMENT OF PATIENT MONITORING AND IDENTIFICATION SYSTEM**" has been carried out by us at Bangalore Institute of Technology as a partial fulfilment of academic requirement of Bachelor of Engineering in Electronics and Instrumentation Engineering under Visvesvaraya Technological University, Belagavi during the academic year 2025-2026.

NAME OF THE STUDENTS	USN	SIGNATURE
ADARSH RAJ	1BI22EI001	
KHUSHI HS	1BI22EI021	
MUSKAAN HUSAIN	1BI22EI029	

Date:

Place: Bengaluru

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without mentioning the people who made it possible, whose constant guidance and encouragement crowned the efforts with success.

We are deeply grateful to our esteemed guide, **Prof. NEETHASHREE N R**., Assistant Professor, Department of Electronics and Instrumentation Engineering, BIT.

We are deeply grateful to our project coordinator, **Dr. ASWATHAPPA P**., Associate Professor and Head, Department of Electronics and Instrumentation Engineering, BIT. His invaluable guidance, unwavering support, and constant encouragement were instrumental in the successful completion of our course.

We would like to acknowledge **Dr. SHANTHAL S**, Principal, BIT, for her invaluable support during this endeavor and timely supports in all forms.

This acknowledgment will be incomplete without thanking our Parents for their support and co-operation and even thanking teaching and non- teaching staff of Department of Electronics and Instrumentation Engineering, BIT for their co-operation and contribution either directly or indirectly.

We wish to express our profound gratitude to everyone who provided their valuable time and assistance in helping us to bring out the best.

ADARSH RAJ	1BI22EI001
KHUSHI HS	1BI22EI021
MUSKAAN HUSAIN	1BI22EI029

ABSTRACT

Design and development of the Patient monitoring and identification system includes integration of facial recognition with real-time vital monitoring to ensure efficient and personalized care. The system utilizes wearable sensors to continuously monitor vital signs such as heart rate, blood pressure, oxygen saturation, temperature, and ECG. A microcontroller unit (ESP32) processes the collected data and employs DSA-based algorithms to detect anomalies. Facial recognition is used for seamless patient identification and to retrieve their complete medical history from a secure database. In the event of critical health conditions, the system triggers instant alerts to healthcare providers. This enables timely intervention and enhances remote patient monitoring. The solution is cost-effective, scalable, and ideal for use in both hospital and home-based care settings.

CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Real time patient monitoring system 2

1.2 Importance of Monitoring and Identification 3

1.3 Real time data collection and analysis 3

1.4 Need for Automation in Healthcare 4

1.5 Benefits for hospitals and medical staff 4

CHAPTER 2 LITERATURE REVIEW AND OBJECTIVES 5

2.1 Literature Review 6

2.2 Objectives 8

CHAPTER 3 SYSTEM ARRANGEMENT 9

3.1 Problem Description 10

3.2 Project Design 10

3.3.1 Block Diagram 10

3.3.2 Functional Diagram 12

3.3 Methodology 13

CHAPTER 4 HARDWARE COMPONENTS 15

4.1 ESP32 Cam module 16

4.2 MAX30102 Pulse oximeter sensor 19

4.3 DS18B20 Digital temperature sensor 21

4.4 Buzzer 23

4.5 USB to TLL converter 24

CHAPTER 5 SOFTWARE REQUIREMENTS 26

5.1 Arduino IDE 27

5.2 Development Environment: ThingsSpeak 29

5.3 Libraries and Dependencies 31

CHAPTER 6 SYSTEM DEVELOPMENT 33

6.1 Circuit Interfacing 34

6.1.1 Interfacing with MAX30102 34

6.1.2 Interfacing with DS18B20 35

6.1.3 Interfacing with Buzzer 35

6.1.4 Interfacing with USB to TLL 36

6.2 Working of the Model 37

CHAPTER 7 RESULT AND DISCUSSION 39

7.1 Circuit Connection 40

7.2 Results 41

7.3 Observations 43

7.4 Existing Model and Developed Model 44

CHAPTER 8 HIGHLIGHTS AND CHALLENGES 45

8.1 Advantages 46

8.2 Limitations 47

8.3 Applications 47

8.4 Future Scope 48

CONCLUSION 49

REFERENCES 50

APPENDIX – I : PROJECT CODE

APPENDIX – II : RESEARCH PAPERS

APPENDIX – III : IEEE PAPER, PUBLICATIONS AND CERTIFICATES

LIST OF FIGURES

Figure 1.1	Real time Patient Monitoring system	2
Figure 3.2.1	Block Diagram of the Developed System	12
Figure 3.2.2	Functional Diagram of the Developed System	13
Figure 3.3	Flowchart of the Developed System	13
Figure 4.1.1	ESP32 cam module	16
Figure 4.1.2	Pinout Diagram of ESP32 cam module	19
Figure 4.2.1	MAX30102 sensor	19
Figure 4.2.2	MAX30102 Pinout diagram	21
Figure 4.3.1	DS18B20 sensor	21
Figure 4.3.2	DS18B20 Pinout Diagram	23
Figure 4.4	Buzzer Pinout Diagram	24
Figure 5.1.1	Arduino IDE	27
Figure 5.1.2	Installation process of Arduino IDE	28
Figure 5.2	ThingsSpeak	29
Figure 5.2.1	Health Monitoring using things Speak	30
Figure 6.1.1	Interfacing with MAX30102	34
Figure 6.1.2	Interfacing with DS18B20	35
Figure 6.1.3	Interfacing with Buzzer	36
Figure 6.1.4	Interfacing with USB to TLL	36
Figure 7.1	Circuit Connection	40
Figure 7.2	Real time Patient Monitoring using ThingsSpeak	42

LIST OF TABLES

Table 2.1	Literature Review	7
Table 7.3	Observations	43
Table 7.4	Existing Model and Developed Model	44

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 REAL TIME PATIENT MONITORING SYTEM

In today's fast-paced healthcare environment, timely and accurate monitoring of patients is essential for improving treatment outcomes and reducing human error. The Patient Monitoring and Identification System is an innovative, IoT-based solution designed to automate the process of identifying patients, monitoring their vital signs, and dispensing medications accordingly. This project integrates computer vision, health sensors, and cloud technologies to create a smart, real-time healthcare assistant.

The system begins by identifying the patient using a **camera module (ESP32-CAM)**, which captures the patient's facial image and retrieves their medical history from a cloud database using facial recognition. Once identified, the system conducts basic health checks using sensors that measure **temperature**, **heart rate**, Based on the real-time health data and the patient's medical history, the system determines the required medication and automatically **dispenses the correct dose** using servo-controlled compartments.

power, and suitable for wearable or embedded applications, making it ideal for long-term monitoring of coma patients.

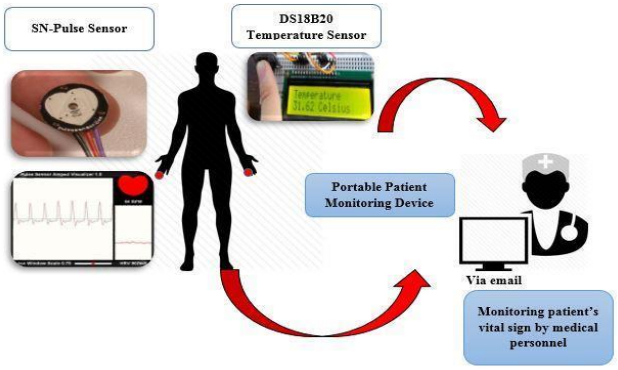


Fig 1.1 Patient monitoring system

1.2 IMPORTANCE OF MONITORING AND IDENTIFICATION

- Continuous monitoring of patient vital parameters
- Accurate identification of patients to avoid medical errors
- Real-time data tracking for timely medical decisions
- Secure storage and access of patient information
- Improved patient safety and quality of healthcare services

1.3 REAL TIME DATA COLLECTION AND ANALYSIS

Real-time data collection and analysis play a crucial role in modern patient monitoring systems. Sensors continuously collect vital parameters such as heart rate, body temperature, oxygen saturation, and blood pressure from the patient. This data is instantly transmitted to a monitoring unit or cloud platform without any delay.

The collected data is analyzed in real time to identify normal and abnormal health conditions. If any parameter crosses the predefined threshold values, the system can immediately alert medical staff. This enables quick response during critical situations and reduces the risk of severe health complications.

Real-time analysis also helps doctors track the patient's condition over a period of time, observe trends, and make accurate treatment decisions. Overall, real-time data collection and analysis improve patient safety, ensure timely medical intervention, and enhance the efficiency of healthcare services.

- Continuous collection of patient vital signs using sensors
- Instant transmission of data to the monitoring system
- Real-time processing and analysis of collected data
- Immediate detection of abnormal or critical values
- Faster medical response and informed decision-making.

1.4 NEED FOR AUTOMATION IN HEALTHCARE

With the increasing number of patients and the growing demand for accurate and timely medical care, traditional manual healthcare systems are becoming insufficient. Automation in healthcare helps in improving efficiency, accuracy, and reliability of medical services by reducing human involvement in repetitive and time-critical tasks. Automation in healthcare has become essential due to the increasing patient load, limited medical staff, and the need for accurate and timely medical services. Traditional manual monitoring methods are time-consuming and prone to human errors, which can affect patient safety. Automated healthcare systems use sensors, embedded devices, and software to continuously monitor patient conditions, collect real-time data, and generate alerts during emergencies. This improves efficiency, reduces the workload on healthcare professionals, enables faster medical response, and ensures better quality of patient care.

1.5 BENEFITS FOR HOSPITAL AND MEDICAL STAFF

Automated patient monitoring and identification systems provide significant benefits to hospitals and medical staff by improving efficiency and workflow management. These systems reduce the need for frequent manual checks, allowing doctors and nurses to focus more on critical patients and complex medical tasks. Real-time access to accurate patient data helps medical staff make faster and better clinical decisions, especially during emergencies. For hospitals, automation improves resource utilization, reduces operational costs, minimizes medical errors, and enhances overall quality of healthcare services, leading to better patient outcomes and improved hospital management.

CHAPTER 2

LITERATURE REVIEW AND OBJECTIVE

CHAPTER 2

LITERATURE REVIEW AND OBJECTIVES

2.1 LITERATURE REVIEW

[1] T. I. Palash, R. Islam and S. Debnath, "Low-Cost Smart COVID-19 Patient Monitoring and Support System," 2021 International Conference on Computer, Communication, Chemical, Materials and Electronic Engineering (IC4ME2), Rajshahi, Bangladesh. 2021

This paper presents a low-cost smart system designed to monitor and support COVID-19 patients remotely. The system focuses on reducing direct contact between patients and healthcare workers to minimize the risk of infection. It uses affordable sensors to measure vital parameters such as body temperature, heart rate, oxygen saturation, and respiration rate. The collected data is processed using a microcontroller and transmitted in real time for monitoring. Alerts are generated when abnormal conditions are detected to ensure timely medical attention.

[2] KH Teoh, RC Ismail, SZM Naziri, R Hussin, MNM Isa, and MSSM Basir. "Face Recognition and Identification using Deep Learning Approach."2022

This paper discusses a deep learning–based approach for face recognition and identification. It focuses on improving accuracy by using neural network models for feature extraction and classification. The proposed method demonstrates effective performance in identifying individuals under varying conditions. Experimental results show that the method can reliably identify and verify individuals, making it suitable for security and healthcare applications.

[3] G. Anita, R.Sankar, A. Vimala Juliet, V. Dhivya, B. Thiyaneshwaran, Ata Kishore Kumar, "Intelligent and Smart Health Care Monitoring System based on Deep Convolutional Neural Network Model. 2023

This paper proposes an intelligent healthcare monitoring system using a deep convolutional neural network model. It automatically analyzes patient data to detect abnormalities and predict health conditions. The system aims to provide accurate, real-time monitoring and support timely medical interventions.

SL. NO.	AUTHORS &YEAR	TITLE	DESCRIPTION	LIMITATIONS
1.	Torikul Islam Palash, Redwanul Islam, Sourav Debnath (2021)	Low-Cost Smart COVID-19 Patient Monitoring and Support System.	A smart healthcare system is designed to remotely monitor patients. This system enhances remote patient care and ensures timely emergency assistance during pandemic situations.	It lacks a reliable patient identification system. Without biometric methods like facial recognition, it cannot accurately track or personalize care for multiple patients
2.	KH Teoh, RC Ismail, SZM Naziri, R Hussin, MNM Isa, and MSSM Basir. (2022)	Face Recognition and Identification using Deep Learning Approach.	The study explores facial recognition methods, their implementation, and real-world applications.	The main limitation of the paper is its reduced accuracy in real-world conditions with poor lighting, occlusions, or background noise.
3.	G. Anita, R.Sankar, A. Vimala Juliet, V. Dhivya, B. Thiyaneshwaran, Ata Kishore Kumar (2023)	Intelligent and Smart Health Care Monitoring System based on Deep Convolutional Neural Network Model.	This research automates the AlexNet model-building process. The proposed model achieves 97.13% accuracy, surpassing existing methods and showcasing its potential as a breakthrough in healthcare.	The system's accuracy may decrease in real-world settings due to factors like inconsistent lighting, sensor noise, or patient movement.

Table 2.1 LITERATURE REVIEW

2.2 OBJECTIVES

The main objective of a patient monitoring system is to ensure continuous and accurate tracking of a patient's health parameters. It aims to provide real-time data to healthcare professionals to improve patient care, reduce errors, and enable timely medical interventions. The system also supports remote monitoring, making healthcare more accessible and efficient. The main objectives are:

1. To continuously monitor vital signs such as heart rate, blood pressure, temperature, and oxygen saturation
2. To detect abnormal or critical health conditions in real time
3. To alert medical staff immediately in case of emergencies
4. To reduce manual monitoring workload for healthcare professionals
5. To maintain accurate patient records for diagnosis and treatment
6. To enable remote monitoring of patients, improving accessibility and safety
7. To support timely and informed medical decision-making

CHAPTER 3

SYSTEM ARRANGEMENT

CHAPTER 3

SYSTEM ARRANGEMENT

3.1 PROBLEM DESCRIPTION

In modern healthcare systems, continuous patient monitoring is a critical requirement, especially for patients in critical care, isolation wards, or those with chronic conditions. Manual monitoring methods are not only labor-intensive but also prone to human error, which can lead to delayed detection of health deterioration. Moreover, accurate patient identification is often challenging in busy hospitals, increasing the risk of administering incorrect treatment or medication. Existing monitoring systems are often expensive and not feasible for small or resource-limited healthcare facilities. Therefore, there is a need for a cost-effective, automated, and reliable patient monitoring and identification system that can track vital signs in real time, alert medical staff during emergencies, and ensure accurate patient identification, ultimately improving healthcare efficiency and patient safety.

3.2 PROJECT DESIGN

a. Hardware Components:

- **ESP32-CAM Module:** Captures patient face for identification and acts as the main processing unit.
- **MAX30102 Sensor:** Measures heart rate and SpO_2 levels.
- **DS18B20 Sensor:** Measures body temperature.
- **Buzzer:** Provides alerts when any vital parameter crosses the safe threshold.

b. Software & Platform:

- **Arduino IDE:** Used for programming the ESP32-CAM and interfacing with sensors.
- **ThingSpeak:** Cloud platform to store, visualize, and analyze patient data in real-time.

c. Data Flow / Workflow:

- Sensors (MAX30102, DS18B20) collect patient vitals → Data sent to ESP32-CAM → ESP32-CAM processes the data and captures the patient image → Data transmitted to

- ThingSpeak → ThingSpeak displays real-time graphs → Buzzer activates for abnormal readings → Face recognition module identifies the patient and retrieves details (name, age, doctor, medicines).

d. Alerts and Monitoring:

- Buzzer alerts medical staff locally for immediate attention.
- Real-time graphs on ThingSpeak allow remote monitoring of patient vitals.

3.2.1 BLOCK DIAGRAM

The block diagram of the Patient Monitoring and Identification System illustrates how all components interact to provide continuous, real-time patient monitoring and identification. At the front end, the system uses sensors including the MAX30102 for heart rate and SpO_2 measurement and the DS18B20 for body temperature measurement. These sensors continuously collect the patient's face for identification purposes. Once the data is processed, it is transmitted via the ESP32's built-in Wi-Fi module to the ThingSpeak cloud platform, where it is stored, analyzed, and displayed as real-time graphs. The system also includes a buzzer connected vital signs from the patient and send the data to the ESP32-CAM module, which acts as the central processing unit. The ESP32-CAM not only processes the incoming sensor data but also captures to the ESP32-CAM, which provides immediate alerts if any of the patient's vital parameters exceed predefined safe limits. For patient identification, the ESP32-CAM uses a face recognition algorithm to match the captured image with stored data and retrieves patient details such as name, age, doctor, and prescribed medicines. This integration of sensors, processing, cloud communication, alerts, and identification ensures continuous monitoring, timely detection of critical conditions, accurate patient identification, and remote accessibility for healthcare providers, thereby improving patient safety and efficiency in hospital management. The system retrieves patient details from a database upon successful face recognition, allowing healthcare professionals to monitor, identify, and respond to patients efficiently, ensuring both safety and timely medical intervention.

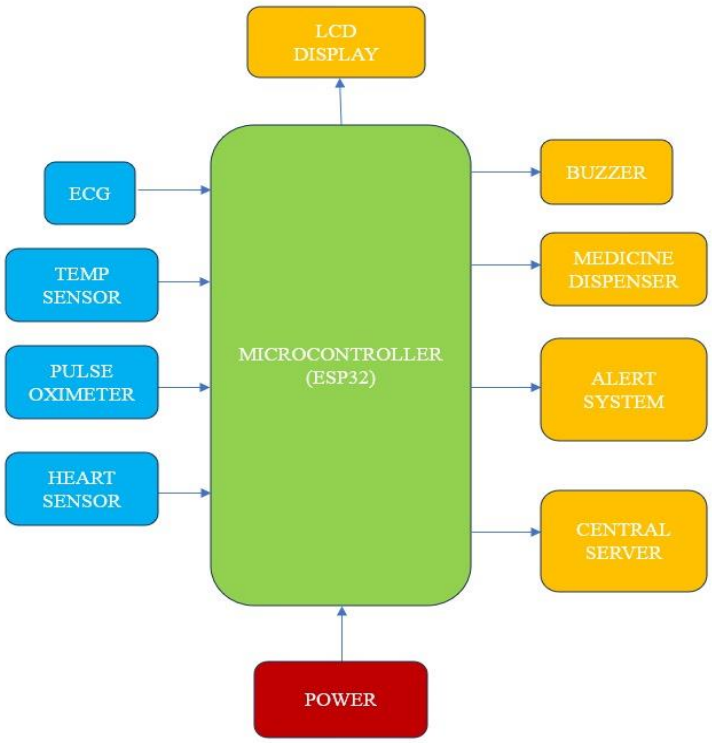


Fig 3.2.1 BLOCK DIAGRAM OF THE DEVELOPED SYSTEM

3.2.2 FUNCTIONAL DIAGRAM

The functional diagram of the Patient Monitoring and Identification System illustrates the flow of information and the roles of each component in the system. The patient is the source of vital signs such as heart rate, SpO_2, and body temperature, as well as the facial image for identification. These parameters are continuously collected by sensors, including the MAX30102 for heart rate and oxygen saturation and the DS18B20 for body temperature, and sent to the ESP32-CAM for processing. The ESP32-CAM analyzes the sensor data to detect any abnormal conditions and performs face recognition by comparing the captured image with stored patient records. If any vital signs exceed safe limits, the buzzer is triggered to alert medical staff immediately.

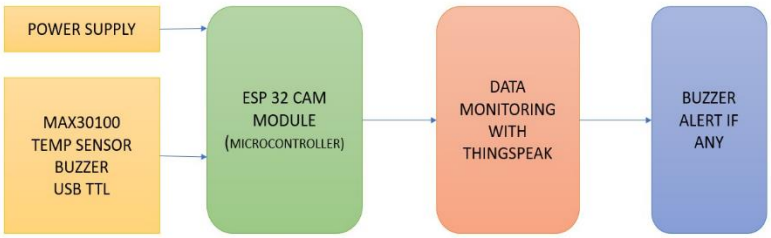


Fig 3.2.2 FUNCTIONAL DIAGRAM OF THE DEVELOPED SYSTEM

3.3 METHODOLOGY

The methodology of the Patient Monitoring and Identification System involves collecting patient vital signs using sensors and processing the data with the ESP32-CAM. Face recognition is used for patient identification, and alerts are generated through a buzzer for abnormal readings. The data is transmitted to the Thing Speak cloud platform for real-time visualization and remote monitoring.

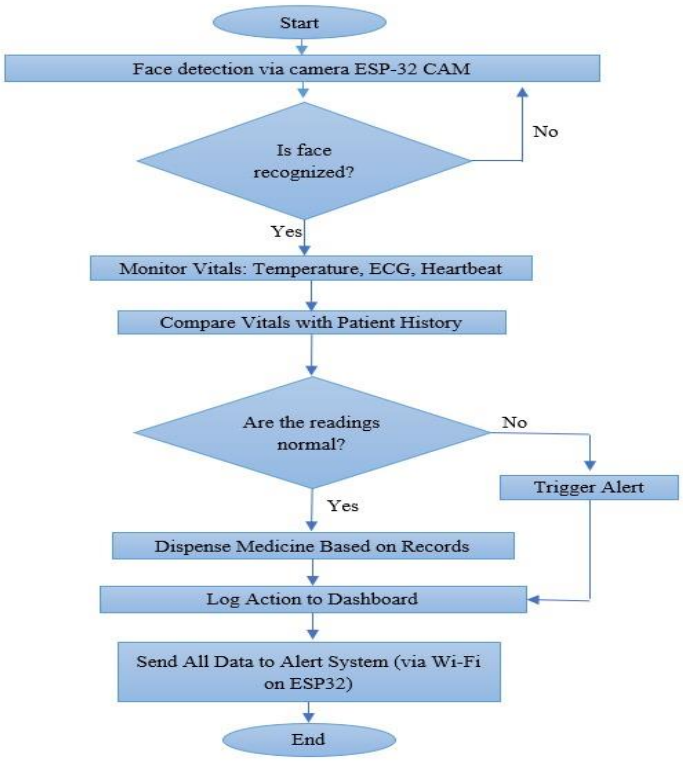


Figure 3.3 FLOWCHART OF THE DEVELOPED SYSTEM

Step 1: Creation of Patient Database

A unique and separate database is created for all registered patients, which stores data like facial identification, previous medical records, vitals, and the medicines prescribed.

Step 2: Design and Development of Model

The model, which holds the camera module for facial detection and devices to check temperature, ECG, and heart rate.

Step 3: Facial Identification of Patient

Upon scanning the face of the patient, all the critical patient data is derived from the database.

Step 4: Monitoring Vitals like Temperature, Heart rate

Through the help of the model, these vitals are measured and compared to previous stored data.

Step 5: Dispense Medicine Based on Records

Based on previously available medical records, the regular dose of medicines is dispensed through the medicine dispenser.

Step 6: Log all actions on the Dashboard

All the activity is reflected on the dashboard of the patient, and it gets registered and logged into the database.

Step 7: Transfer Complete Data Through ESP32/Wi-Fi

The complete data gets transferred via ESP32/Wi-Fi to archives for future reference.

Step 8: Documentation and Reporting

A proper and complete report is prepared, and the model is regularly monitored and maintained to avoid any discrepancies.

CHAPTER 4

HARDWARE COMPONENTS

CHAPTER 4

HARDWARE COMPONENTS

The patient monitoring and identification system uses several hardware components to facilitate health monitoring and movement detection. Each component is specific in ensuring the system's efficiency and accuracy.

4.1 ESP32 CAM MODULE

Fig 4.1 ESP32 Cam module

The ESP32-CAM is a low-cost, compact development board that combines the capabilities of the ESP32 microcontroller with a camera module. It supports Wi-Fi and Bluetooth, making it ideal for wireless image transmission and IoT applications. The board comes equipped with an OV2640 camera sensor, capable of capturing still images and video streams at moderate resolutions.

Key features:

- Microcontroller: ESP32 with dual-core processor and integrated Wi-Fi and Bluetooth
- Camera: OV2640 sensor with up to 2MP resolution
- Connectivity: Supports Wi-Fi (802.11 b/g/n) and Bluetooth (Classic and BLE)
- Storage: MicroSD card slot for storing images or video locally

Applications in this project:

- Used for face detection and recognition to identify patients
- Captures live images and sends them to a server or cloud for processing
- Provides a contactless method for accurate patient verification
- Enables real-time communication with other ESP32 modules for triggering sensors or medicine dispensing

Advantages:

- Low cost and power-efficient, ideal for portable or embedded systems
- Compact design makes it easy to integrate into small healthcare devices
- Can operate in standalone mode or as part of a larger IoT system

How ESP32 is Better Than Previous Microcontrollers:

The ESP32-CAM module is a compact, low-cost development board based on the ESP32 microcontroller with an integrated OV2640 camera. It features built-in Wi-Fi connectivity, enabling wireless transmission of images and data without the need for additional modules. The module supports image capture, video streaming, and basic image processing, making it suitable for applications such as face detection and identification. Despite its small size, it offers sufficient processing power and GPIO support for interfacing with sensors. Due to its affordability, versatility, and camera capability, the ESP32-CAM is widely used in IoT, surveillance, and smart healthcare monitoring systems.

ESP32 CAM MODULE PIN FUNCTIONS

5V – Provides the main power supply to the ESP32-CAM module and the camera unit. A stable

5V input is required to ensure reliable operation and proper image capture.

GND – Serves as the common ground reference for the ESP32-CAM and all connected external components, ensuring correct circuit operation.

3.3V – Outputs a regulated 3.3V supply that can be used to power low-current sensors and peripherals connected to the module.

GPIO 0 (IO0) – Used to place the ESP32-CAM into flashing mode for program uploading. It must be grounded during code upload and left floating during normal execution.

GPIO 1 (U0TXD) – Acts as the UART transmit pin, allowing the ESP32-CAM to send debugging messages and system data to the serial monitor.

GPIO 3 (U0RXD) – Functions as the UART receive pin, enabling the ESP32-CAM to receive data from external devices or a computer.

GPIO 2 – A general-purpose input/output pin that can be configured for digital input or output based on application requirements.

GPIO 4 – Commonly used as a digital output pin and suitable for controlling alert devices such as buzzers or LEDs.

GPIO 12 – Works as a general-purpose I/O pin and can be used for sensor interfacing or digital signal input.

GPIO 13 – Provides flexible digital input/output functionality for connecting external peripherals or modules.

GPIO 14 – Often used for communication protocols such as SPI or for interfacing sensors that require clocked data transfer.

GPIO 15 – A general-purpose I/O pin that must be used carefully, as it plays a role in the ESP32-CAM boot configuration.

GPIO 16 – Functions as a digital I/O pin and can be used for additional external device connections when available.

Fig 4.1.2. Pinout Diagram of ESP32 Cam module

4.2 MAX30102 PULSE OXIMETER SENSOR

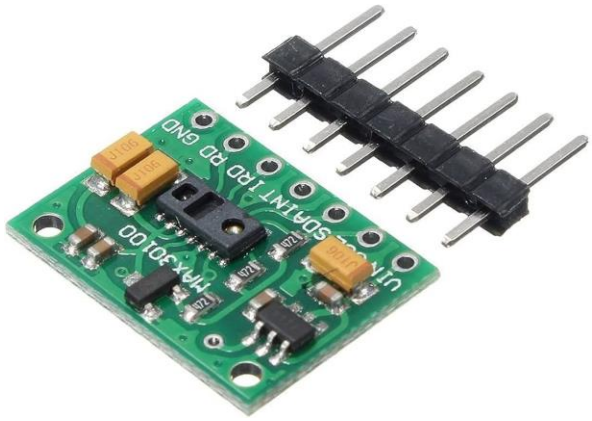


Fig 4.2.1MAX30100

The MAX30100 is an integrated sensor module that combines pulse oximetry and heart-rate monitoring capabilities. It includes two LEDs (red and infrared), a photodetector, and signal processing components to measure oxygen saturation (SpO_2) and pulse rate accurately. The sensor communicates via the I2C interface with microcontrollers such as the ESP32, enabling easy integration into various health monitoring systems. It offers continuous, non-invasive monitoring of heart rate and blood oxygen levels, which is important for detecting conditions like hypoxia and irregular heartbeats. The module is compact, low-power, and suitable for wearable or portable medical device.

Key features

- Combines pulse oximetry and heart rate monitoring in a single module
- Uses two LEDs (red and infrared) and a photodetector for accurate SpO_2 and pulse measurements
- Communicates with microcontrollers via I2C interface
- Low power consumption, ideal for portable and wearable devices
- Compact size suitable for integration into small health monitoring systems
- Built-in ambient light cancellation for improved signal accuracy

MAX30102 PIN FUNCTIONS

VIN – Supplies power to the MAX30102 module. It can typically accept 3.3V or 5V depending on the breakout board design.

GND – Acts as the ground reference for the sensor and must be connected to the system ground.

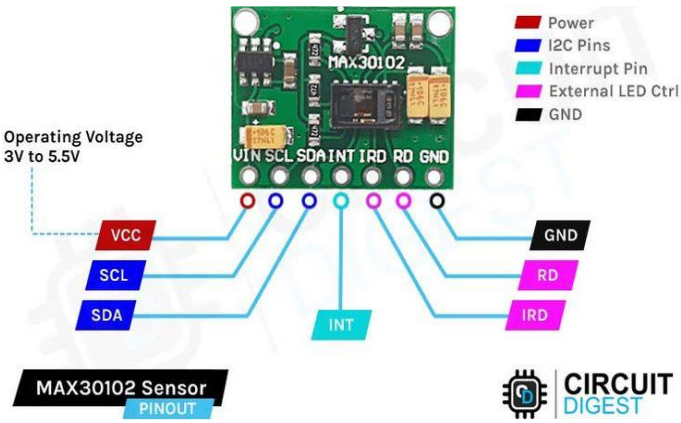
SCL – Serial Clock Line used in I^2C communication to synchronize data transfer between the MAX30102 and the ESP32-CAM.

SDA – Serial Data Line used in I^2C communication to transmit heart rate and SpO_2 data between the sensor and the microcontroller.

INT – Interrupt pin used to notify the microcontroller when new sensor data is available or when specific events occur.

IR – Infrared LED pin used internally to emit infrared light for detecting blood oxygen saturation levels.

RED – Red LED pin used internally to emit red light required for heart rate and SpO_2 measurement.



4.2..2 MAX30102 PINOUT DIAGRAM

4.3 DS18B20 DIGITAL TEMPERATURE SENSOR

Figure 4.3.1 DS18B20 SENSOR

The DS18B20 is a digital temperature sensor that provides accurate temperature readings in a digital format. It uses the 1-Wire communication protocol, which allows data transfer using only a single data line, reducing wiring complexity. The sensor directly converts temperature measurements into digital values, eliminating the need for external analog-to-digital converters. Due to its reliability, accuracy, and ease of interfacing, the DS18B20 is widely used in healthcare, industrial, and environmental monitoring applications.

Key Features of DS18B20

- Measures temperature in the range of −55°C to +125°C
- Provides digital temperature output with high accuracy
- Uses 1-Wire communication protocol
- Requires only one data pin for communication
- Can be powered in normal or parasite power mode
- Each sensor has a unique 64-bit ID for identification

Working Principle:

The DS18B20 operates by sensing the ambient or body temperature through its internal temperature-sensing element. The sensor converts the measured temperature into a digital value using an internal analog-to-digital converter. This digital temperature data is transmitted to the microcontroller through the 1-Wire communication protocol, which requires only a single data line. The microcontroller sends a command to the DS18B20 to initiate temperature conversion, after which the sensor processes the data and sends the temperature value in digital format. This eliminates the need for external signal conditioning or ADC, making the DS18B20 accurate, reliable, and easy to interface with embedded systems.

DS18B20 Pin Functions

VDD – Supplies power to the DS18B20 sensor, typically 3.3V or 5V depending on the system design.

DQ – Data pin used for 1-Wire communication to transmit and receive temperature data between the sensor and microcontroller.

GND – Ground pin that provides the reference voltage and completes the electrical circuit.

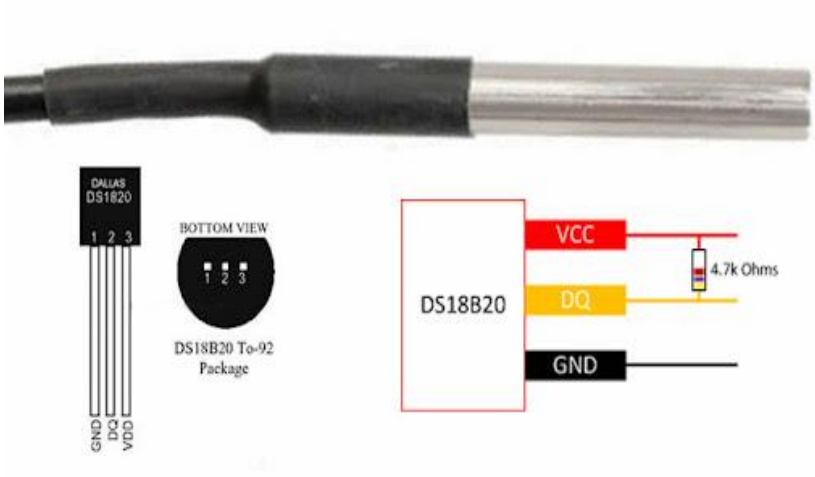


Figure 4.3.2 DS18B20 SENSOR PINOUT

4.4 BUZZER

A buzzer is an audio signaling device used to provide audible alerts or warnings. It converts electrical energy into sound energy using either piezoelectric or electromagnetic principles. In patient monitoring systems, a buzzer is used to alert medical staff when vital parameters exceed safe limits. Its simple operation and quick response make it effective for emergency indications.

Features:

- Provides immediate audible alerts
- Simple and reliable operation
- Low power consumption
- Easy to interface with microcontrollers
- Suitable for warning and notification systems

WORKING PRINCIPLE

A buzzer works by converting electrical energy into sound energy to provide an audible alert. When a voltage signal is applied from the microcontroller to the buzzer, it causes the internal element (piezoelectric or electromagnetic) to vibrate and produce sound. In this project, the ESP32-CAM controls the buzzer by sending a digital HIGH signal when any monitored vital parameter exceeds the predefined safe limits. The buzzer remains active until the condition returns to normal or the alert is reset, thereby providing an immediate warning to medical staff.

Buzzer Pin Functions

VCC – Supplies power to the buzzer, usually from a 3.3V or 5V source depending on the buzzer type.

GND – Ground pin that completes the electrical circuit and provides reference voltage.

SIGNAL – Control pin connected to the microcontroller GPIO, used to turn the buzzer ON or OFF.

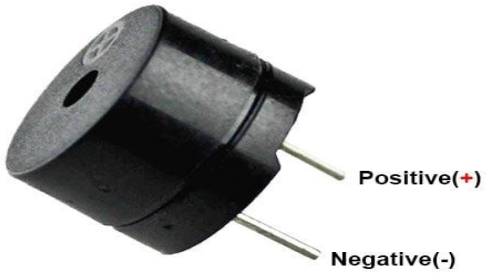


Fig 4.4 Buzzer Pinout diagram

4.5 USB TO TLL CONVERTER

Fig 4.5 USB TO TLL Converter

A USB to TTL converter is used to enable serial communication between a computer and a microcontroller that operates using TTL logic levels. It converts USB signals from the computer into serial UART signals that the microcontroller can understand. In this project, the USB to TTL converter is mainly used for uploading programs to the ESP32-CAM and for viewing serial monitor output during debugging

USB to TTL Converter Pin Functions

VCC – Provides power output (3.3V or 5V depending on the module) to external devices if required.

GND – Ground pin that provides a common reference between the converter and microcontroller.

TXD – Transmit pin used to send serial data from the USB converter to the microcontroller RX pin.

RXD – Receive pin used to receive serial data from the microcontroller TX pin.

CTS – Clear to Send pin used for hardware flow control (optional and often unused).

RTS – Request to Send pin used for hardware flow control or reset control in some applications.

Key Features of USB to TTL Converter

- Enables USB to serial communication
- Supports UART communication protocol
- Compatible with microcontrollers like ESP32, Arduino, and others
- Provides stable data transfer for programming and debugging
- Simple and easy to use

Working Principle of USB to TTL Converter

The USB to TTL converter works by converting USB signals from a computer into serial UART signals compatible with microcontrollers. When connected to a computer, the converter is recognized as a serial communication device. Data sent from the computer through the USB interface is converted into TTL-level serial data and transmitted through the TX pin to the microcontroller's RX pin. Similarly, data sent from the microcontroller through its TX pin is converted by the converter and sent back to the computer via USB. This allows reliable program uploading, debugging, and serial communication between the computer and the ESP32-CAM.

CHAPTER 5

SOFTWARE REQUIREMENTS

CHAPTER 5

SOFTWARE REQUIREMENTS

5.1 Arduino Integrated Development Environment (IDE)

The Arduino Integrated Development Environment - or Arduino Software (IDE) contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

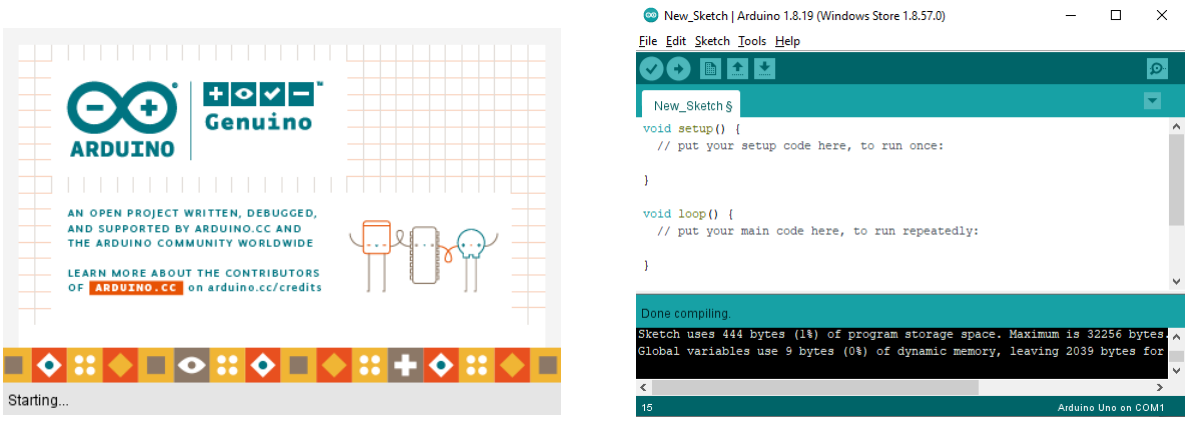


Figure 5.1.1 Arduino IDE

1. **Text Editor:** Features include creating, opening, and saving sketches, with capabilities for text manipulation like cutting/pasting and search/replace.

2. **Message Area:** Provides feedback, displays errors, and gives updates while saving/exporting sketches.

3. **Console:** Showcases detailed error messages and other related information.

4. **Serial Port Display:** Located at the window's bottom right corner. This displays serial sent from the Arduino board over a USB or serial connector and sends data to the board.

5. **Toolbar Features:** Buttons available for functions like verifying and uploading sketches, and accessing the serial monitor.

6. **Libraries:** Provide extra functionality for use in sketches

Arduino Software (IDE) includes the built-in support for the boards like Arduino Due, Arduino Zero, Edison, Galileo and so on

INSTALLATION FOR ARDUINO IDE

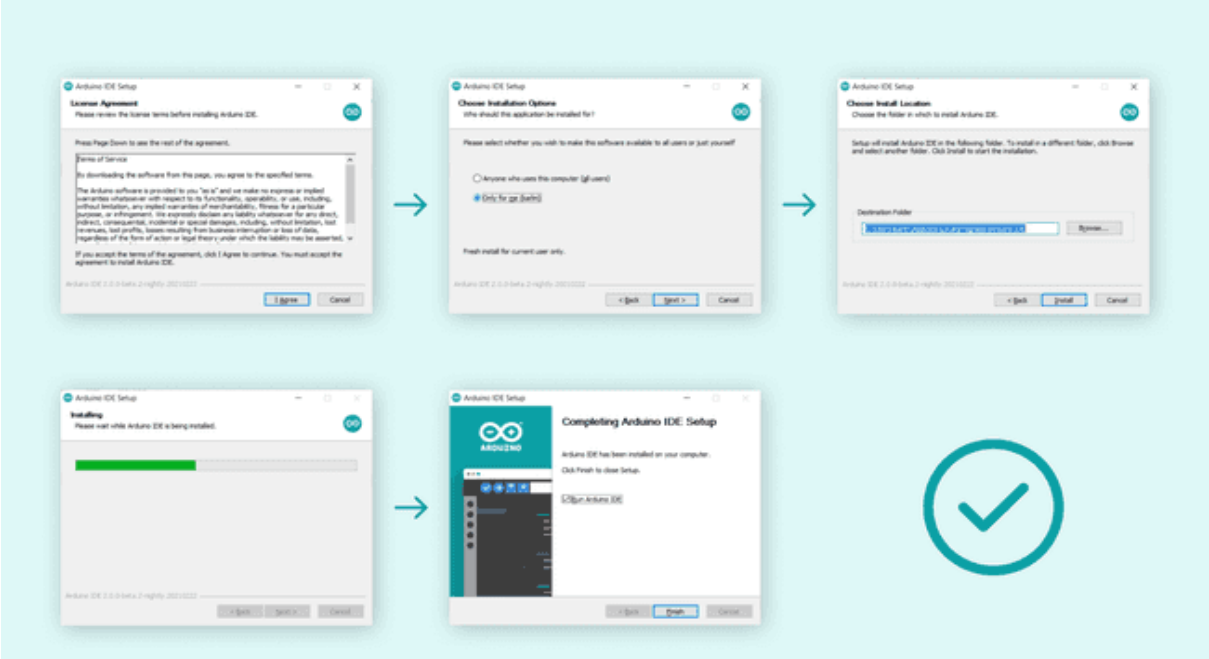


Fig 5.1.2 Installation process of Arduino IDE

Step 1 – Download the Arduino IDE from the official Arduino website suitable for your operating system to ensure compatibility and stability.

Step 2 – Install the Arduino IDE by running the downloaded setup file and following the on-screen instructions to complete the installation.

Step 3 – Launch the Arduino IDE after installation to verify that the software has been installed correctly.

Step 4 – Open the Boards Manager from the Tools menu to add support for additional microcontroller boards such as ESP32.

Step 5 – Add the ESP32 board URL in the Preferences section to enable ESP32 board installation.

Step 6 – Install the ESP32 board package from the Boards Manager to allow programming of the ESP32-CAM module.

Step 7 – Connect the USB to TTL converter to the computer and ensure the required drivers are installed for serial communication.

Step 8 – Select the correct board and COM port from the Tools menu to establish communication between the IDE and the ESP32-CAM.

5.2 DEVELOPMENT ENVIRONMENT: THINGS SPEAK

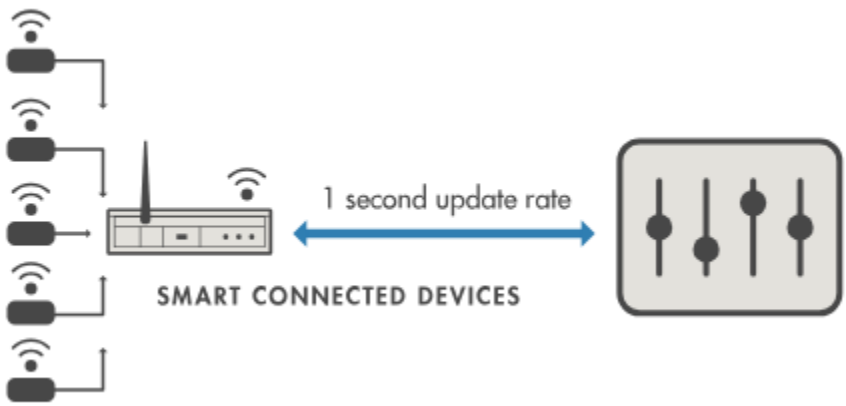


Fig 5.2 ThingsSpeak

ThingSpeak is used as a cloud-based IoT platform to store, analyze, and visualize patient health data in real time. The ESP32-CAM sends sensor data such as heart rate, SpO_2, and body temperature to ThingSpeak through Wi-Fi. ThingSpeak displays this data in the form of live graphs, enabling doctors and caregivers to remotely monitor the patient's condition. It also helps in maintaining historical health records, which can be useful for analysis and medical decision-making.

Features of ThingSpeak

1. Real-time data storage and visualization
2. Live graphical representation of sensor data
3. Cloud-based access from anywhere
4. Support for IoT devices using HTTP and MQTT
5. Data analysis using built-in tools
6. Secure data handling with API keys
7. Easy integration with microcontrollers like ESP32

ROLE OF THINGS SPEAK

ThingSpeak acts as the cloud-based backbone of the patient monitoring system by enabling real-time data transmission, storage, and visualization. It receives continuous health data from the ESP32-CAM, including heart rate, SpO_2, and body temperature, through secure API-based communication. The platform converts the incoming data into live graphical representations, making it easier for doctors and caregivers to observe changes in the patient's condition over time. ThingSpeak also stores historical records of patient data, which can be used for trend analysis and medical evaluation. By enabling remote access to patient information from anywhere, ThingSpeak reduces the need for constant physical monitoring and improves the efficiency and reliability of healthcare services.

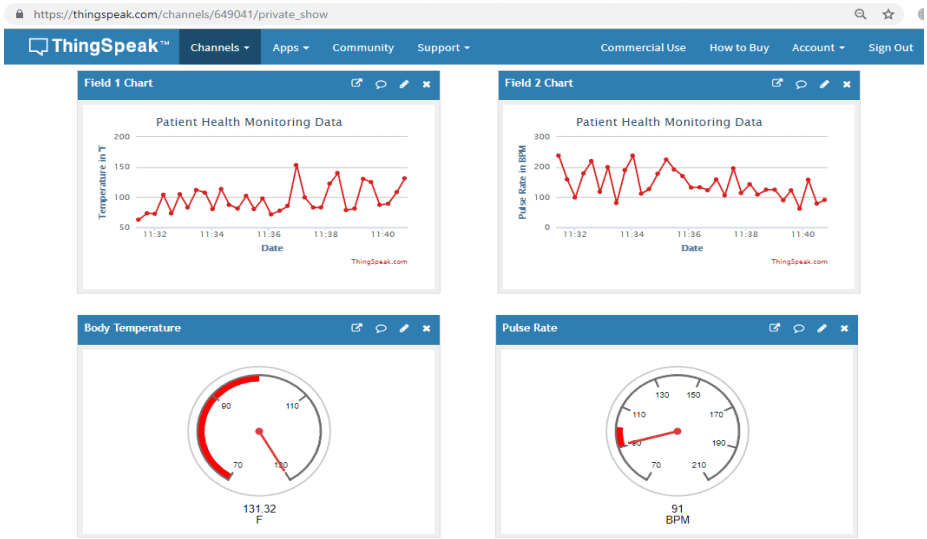


Fig 5.2.1 Health Monitoring using ThingsSpeak

1. Create a ThingSpeak account and set up a dedicated channel for patient monitoring to organize and manage health data.
2. Define separate fields in the channel for parameters such as heart rate, SpO_2, and body temperature to ensure structured data storage.
3. Generate a unique Write API key, which is used to securely send sensor data from the ESP32-CAM to the ThingSpeak server.
4. Program the ESP32-CAM using Arduino IDE to connect to a Wi-Fi network for internet communication.

5. Use HTTP requests in the program to transmit sensor readings to ThingSpeak at regular intervals.
6. ThingSpeak receives and stores the incoming data in its cloud database for further use.
7. The platform displays the data in the form of real-time graphs, making it easy to observe changes in patient health.
8. Doctors and caregivers can remotely access both live and historical data for continuous monitoring and analysis.

5.3 LIBRARIES AND DEPENDENCIES

This project uses several software libraries to enable communication between the ESP32-CAM, sensors, and the cloud platform. These libraries simplify hardware interfacing, data processing, and internet communication, allowing efficient development of the patient monitoring and identification system. Libraries and dependencies play a vital role in the successful implementation of this project, as they provide predefined functions and frameworks required for hardware interfacing, data communication, and system control. Instead of writing complex low-level code, libraries allow efficient and reliable interaction between the ESP32-CAM, sensors, and cloud platforms. Dependencies ensure compatibility between different software components and manage communication protocols such as Wi-Fi, I^2C, 1-Wire, and HTTP. By using appropriate libraries, the development process becomes faster, code readability is improved, and system stability is enhanced. These libraries collectively support real-time data acquisition, patient identification, alert generation, and cloud-based monitoring in the patient monitoring system. Some of the libraries used are as follows:

The **WiFi library** is used to establish a wireless connection between the ESP32-CAM and the internet. It enables the microcontroller to connect to a Wi-Fi network and transmit patient data to the ThingSpeak cloud platform for remote monitoring.

The **HTTPClient library** is used to send sensor data to ThingSpeak using HTTP requests. It helps in formatting and transmitting data securely through API keys, allowing real-time data upload and cloud storage.

The **Wire library** is required for I^2C communication between the ESP32-CAM and the MAX30102 sensor. It enables reliable data exchange over the SDA and SCL lines for heart rate and SpO_2 measurement.

The **MAX3010x / MAX30102 sensor library** provides predefined functions to initialize the sensor, read heart rate, and calculate oxygen saturation. It simplifies sensor configuration and data extraction without requiring low-level programming.

The **OneWire library** is used to support 1-Wire communication with the DS18B20 temperature sensor. It allows the ESP32-CAM to communicate with the sensor using a single data pin.

The **DallasTemperature library** works along with the OneWire library to convert raw temperature data from the DS18B20 into readable temperature values in degrees Celsius.

The **ESP32 Camera (esp_camera) library** is used to control the camera module integrated with the ESP32-CAM. It enables image capture and supports camera initialization for face detection and identification tasks.

The **Arduino core libraries** provide basic functions such as pin control, timing, and serial communication. These core libraries form the foundation for program execution, debugging, and system control.

Together, these libraries and dependencies ensure smooth integration of sensors, camera, Wi-Fi communication, cloud data transfer, and alert mechanisms, making the overall patient monitoring and identification system reliable and efficient.

CHAPTER 6

SYSTEM DEVELOPMENT

CHAPTER 6

SYSTEM DEVELOPMENT

6.1 CIRCUIT INTERFACING

Circuit interfacing involves connecting the ESP32-CAM with sensors and peripherals to enable data collection and alerts. The MAX30102 is interfaced using the I²C protocol to measure heart rate and SpO_2, while the DS18B20 uses the 1-Wire protocol to measure body temperature. A buzzer is connected to a GPIO pin to provide audible alerts during abnormal conditions. The USB to TTL converter is used for programming the ESP32-CAM and serial communication with the computer.

6.1.1 INTERFACING WITH MAX30102

Purpose: To measure a patient's heart rate and blood oxygen (SpO_2) levels.

Connections:

- VIN – 3.3V
- GND - GND
- SCL – GPIO 15
- SDA – GPIO 14

Communication: With the ESP32 using the **I²C protocol** via SDA and SCL lines..

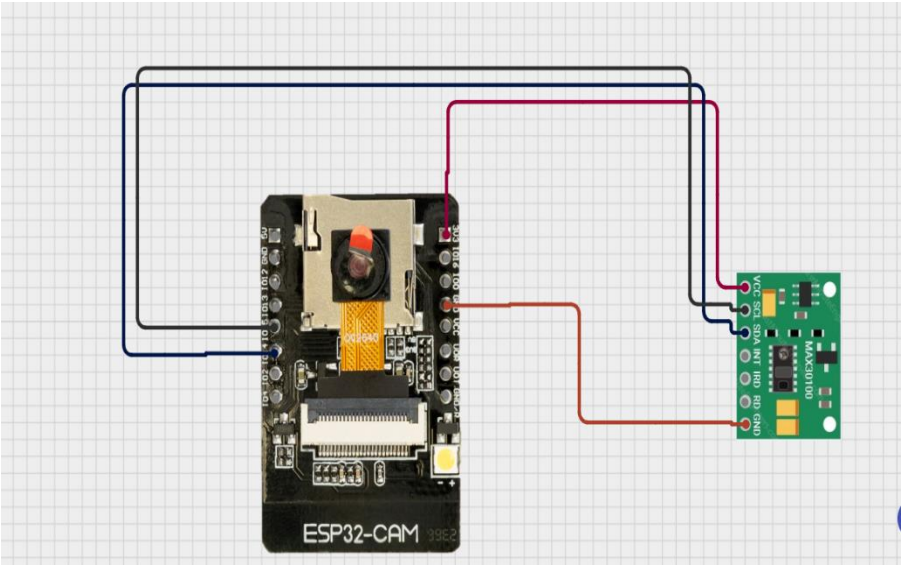


Figure 6.1.1 INTERFACING WITH MAX30102

6.1.2 INTERFACING WITH DS18B20

Purpose: To measure Body temperature.

Connections:

- DATA – GPIO 2
- VCC – 3.3 V
- GND - GND.

Communication: Use the **One-Wire protocol** through a single data pin.

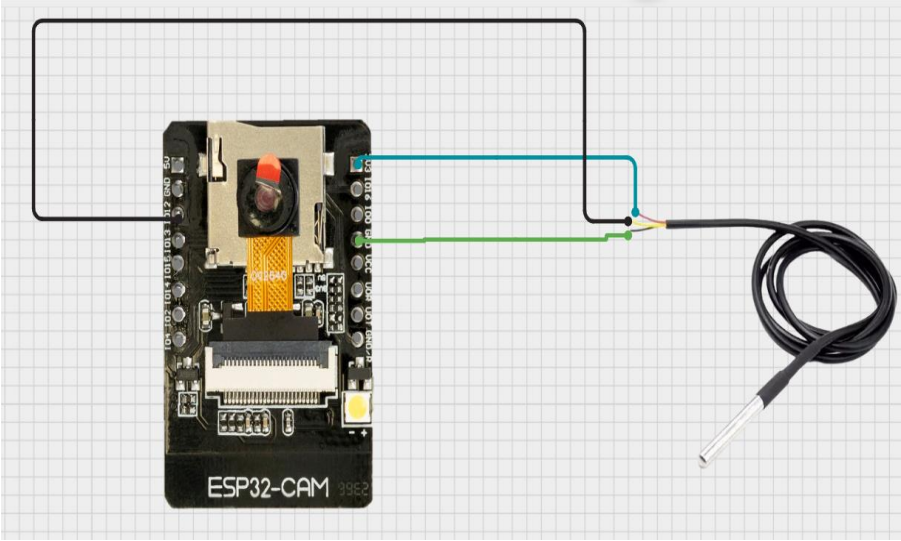


Figure 6.1.2 INTERFACING WITH DS18B20

6.1.3 INTERFACING WITH BUZZER

Purpose: The purpose of the buzzer is to provide an immediate audible alert when the ESP32-CAM detects abnormal or critical conditions in the patient's vital parameters.

Connections:

- + Positive - GPIO 16
- - Negative - GND.

Communication: The buzzer communicates with the ESP32-CAM through a GPIO pin, where the ESP32-CAM sends a digital control signal that activates the buzzer to produce an audible alert.

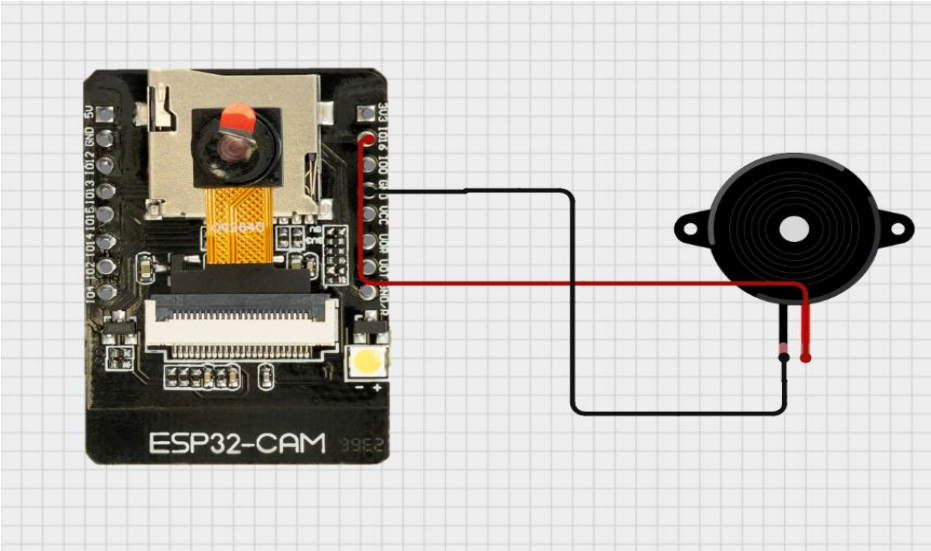


Fig 6.1.3 Interfacing with Buzzer

6.1.4 INTERFACING WITH USB TO TLL CONVERTER

Purpose: The USB to TTL converter is used to upload programs to the ESP32-CAM and to enable serial communication between the ESP32-CAM and the computer.

Connections:

- DATA – GPIO 27
- VCC - 3.3V
- GND - GND.

Communication: Through UART serial communication using TX and RX pins.

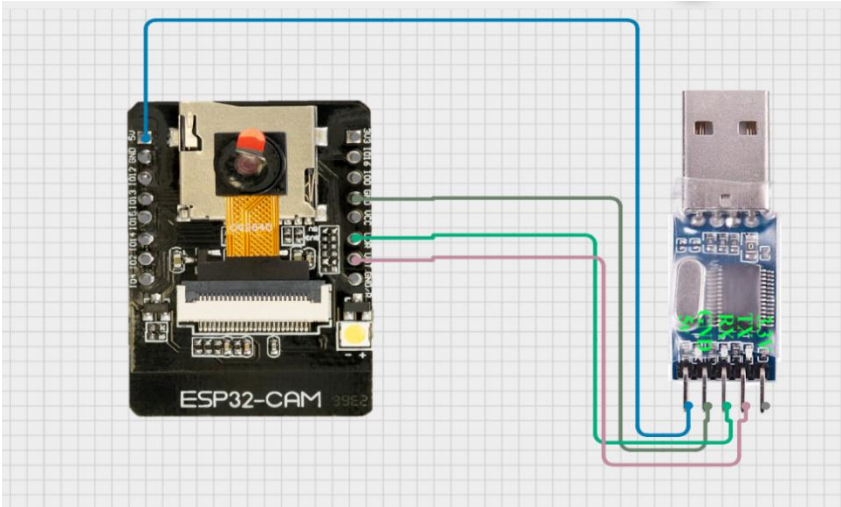


Fig 6.1.4 Interfacing with USB to TLL converte

6.2 WORKING OF THE MODEL

The final complete model of the patient monitoring and identification system operates by integrating sensing, processing, alerting, identification, and cloud monitoring functions to ensure effective healthcare support. The overall working can be explained as follows:

- The process begins with the sensors placed on the patient, where the MAX30102 continuously measures heart rate and oxygen saturation, and the DS18B20 measures body temperature. This ensures real-time acquisition of vital health parameters without manual intervention.
- The collected sensor data is transmitted to the ESP32-CAM, which acts as the central processing and control unit of the system. It receives, processes, and organizes all incoming data for further analysis.
- The ESP32-CAM compares the measured values with predefined safe threshold limits to determine the patient's health condition. This real-time analysis helps in identifying abnormal or critical situations at an early stage.
- When any vital parameter exceeds the normal range, the ESP32-CAM immediately activates the buzzer, providing an audible alert to nearby medical staff for quick attention and response.
- Along with health monitoring, the ESP32-CAM captures the patient's facial image and performs patient identification, ensuring that the recorded data is linked to the correct individual and reducing the chances of medical errors.
- After processing, the ESP32-CAM transmits the patient's vital data and identification information to the ThingSpeak cloud platform using Wi-Fi.
- ThingSpeak stores the incoming data and displays it in the form of real-time graphs, allowing doctors and caregivers to remotely monitor the patient's condition and analyze health trends over time.
- Overall, this integrated working model improves patient safety, enables timely medical intervention, and reduces the workload on healthcare professionals by providing continuous, automated, and remote monitoring.

In conclusion, the final complete model of the patient monitoring and identification system plays a significant role in improving modern healthcare services. By continuously monitoring vital parameters such as heart rate, SpO_2, and body temperature, the system ensures early detection of abnormal health conditions. The integration of patient identification using the ESP32-CAM reduces the risk of medical errors and ensures accurate association of health data with the correct patient. Real-time alerts through the buzzer enable quick medical response, while cloud-based monitoring using ThingSpeak allows doctors and caregivers to access patient data remotely. Overall, the system enhances patient safety, improves efficiency in healthcare delivery, reduces manual workload, and provides a reliable, low-cost solution for smart healthcare monitoring.

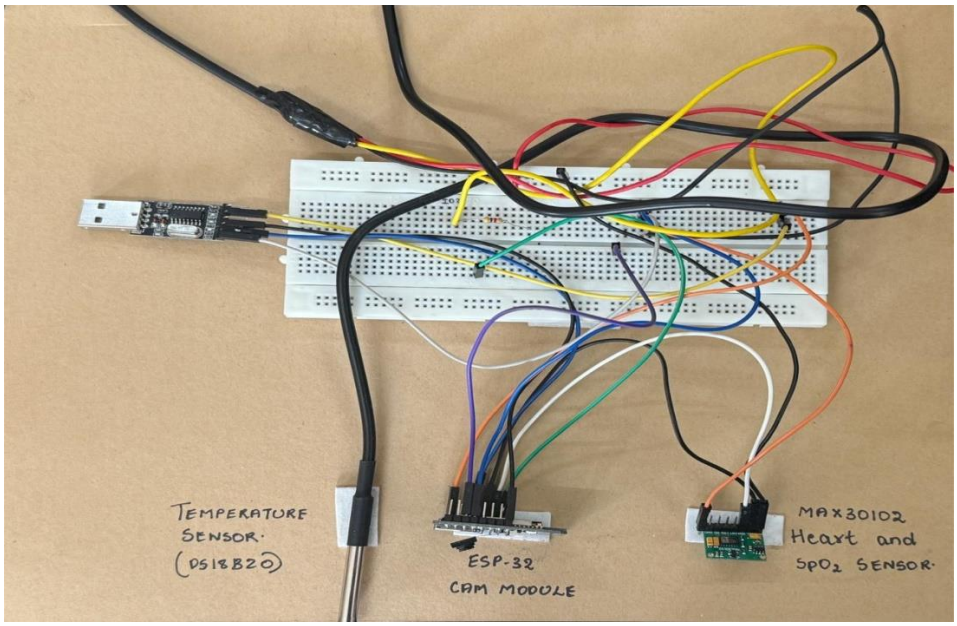


Figure 6.2 DEVELOPED MODEL

CHAPTER 7

RESULT AND DISCUSSION

CHAPTER 7

RESULT AND DISCUSSION

7.1 CIRCUIT CONNECTION

The circuit connection of the system is designed to ensure reliable communication between the ESP32-CAM and all peripheral components. Sensors such as the MAX30102 and DS18B20 are interfaced using standard communication protocols to accurately collect patient data. The buzzer is connected to a GPIO pin of the ESP32-CAM to provide immediate alerts during abnormal conditions. Proper power supply and common grounding are maintained across all components to ensure stable and error-free operation.

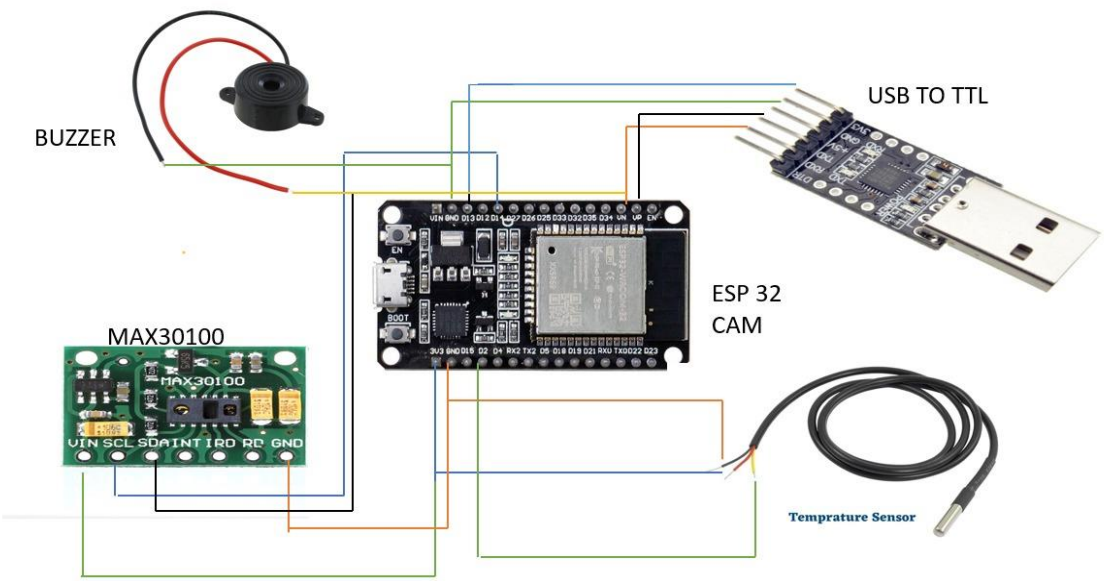


Figure 7.1 CIRCUIT CONNECTION

The diagram represents the interconnection of various components with the ESP32 CAM MODULE:

ESP32-CAM: The ESP32-CAM acts as the main controller of the system, processing sensor data, controlling the buzzer, capturing images for patient identification, and transmitting data to the cloud using Wi-Fi.

MAX30100 Sensor: The VIN pin is connected to 3.3V and GND to GND of the ESP32-CAM, while SDA and SCL are connected to the I^2C GPIO pins, enabling heart rate and SpO_2 measurement.

DS18B20 Temperature Sensor: The VDD pin is connected to 3.3V, GND to GND, and the DATA (DQ) pin to a GPIO pin of the ESP32-CAM, allowing temperature measurement using the 1-Wire protocol.

Buzzer: The positive terminal of the buzzer is connected to a GPIO pin and the negative terminal to GND, providing an audible alert when abnormal patient conditions are detected.

USB to TTL Converter: The TXD pin is connected to RX (GPIO 3) and RXD to TX (GPIO 1) of the ESP32-CAM, with GND connected to GND, enabling program uploading and serial monitoring.

Programming Mode Connection: The GPIO 0 pin is connected to GND during code uploading and disconnected for normal operation.

Power Supply and Ground: All components share a common ground and are powered using a stable 3.3V/5V supply, ensuring reliable and stable system operation.

7.2 RESULTS

- The implemented system demonstrated effective **real-time monitoring of vital health parameters**, including **heart rate, SpO_2, and body temperature**, using the MAX30102 and DS18B20 sensors. The readings were consistent and suitable for continuous healthcare observation.

- The **ESP32-CAM** efficiently processed all incoming sensor data and acted as a reliable central controller. It handled data acquisition, threshold comparison, patient identification, and wireless data transmission without system lag.

- The **buzzer alert system** functioned effectively by providing immediate audible warnings whenever any vital parameter exceeded the predefined safe limits. This ensured quick attention and timely medical response during abnormal conditions.

- **Patient identification** using the ESP32-CAM camera ensured that health data was linked to the correct individual. This minimized the risk of data mismatch and improved reliability in multi-patient monitoring environments.

- Through **ThingSpeak monitoring**, sensor data was successfully uploaded to the cloud in real time using Wi-Fi. The platform displayed **live, continuously updating graphs**, allowing easy visualization of patient health status.

- **Historical data storage** on ThingSpeak enabled trend analysis, helping in observing gradual changes in patient health over time and supporting better medical decision-making.

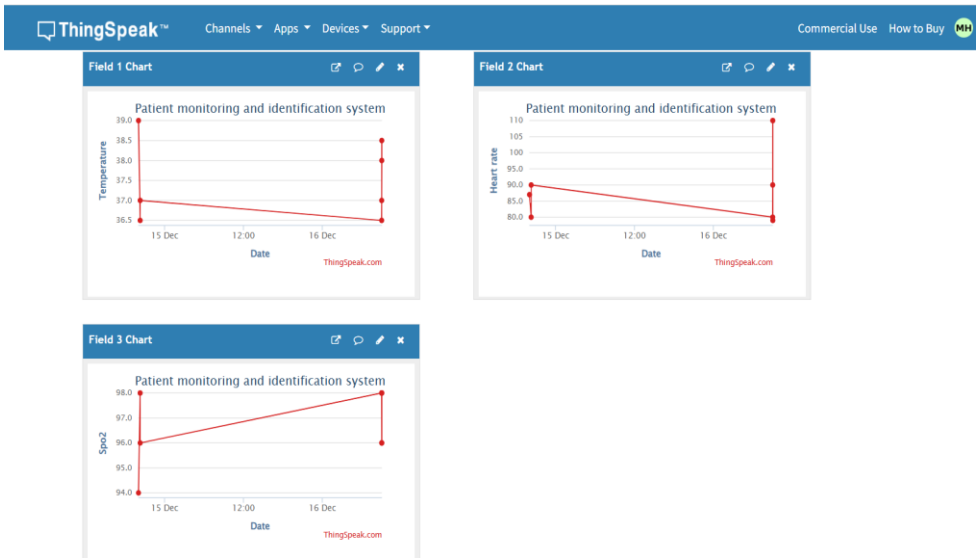


Figure 7.2 Real Time patient monitoring using ThingsSpeak

7.3 OBSERVATIONS

The observation phase focuses on analyzing the performance of the patient monitoring and identification system during real-time operation. The recorded sensor data and cloud-based outputs were examined to evaluate accuracy, reliability, and system responsiveness.

Table 7.3 OBSERVATIONS

PARAMETERS	MINIMUM VALUE	MAXIMUM VALUE	TEST SAMPLE 1	TEST SAMPLE 3
Temperature (°C)	36.5	39.0	37.0	38.0
Heart Rate (BPM)	80	110	90	105
SpO_2 (%)	94	98	96	98

Analysis of Observations

- The system demonstrated **reliable real-time monitoring** of vital parameters such as **body temperature**, **heart rate**, and **SpO_2** without data loss.
- The observed values remained within **normal physiological ranges**, and abnormal variations were clearly identified by the system.
- The **ThingSpeak cloud platform** successfully displayed **live, continuous graphs**, allowing easy visualization of patient health trends.
- The system showed **consistent data transmission** from the ESP32-CAM to the cloud with minimal delay.
- The **alert mechanism** responded effectively whenever predefined threshold limits were crossed, ensuring timely attention.
- Overall, the observations confirm that the system provides **accurate monitoring**, **remote accessibility**, and **efficient performance** for smart healthcare applications

7.3 EXISTING AND DEVELOPED MODEL

FACTORS	EXISTING MODEL	DEVELOPED MODEL
Patient Identification	Uses RFID cards, QR codes, or manual input. Some costly systems use fingerprint or iris scanners. These methods are contact-based and can cause identification errors.	Uses face recognition via ESP32-CAM, which is contactless, real-time, and low-cost. It improves hygiene and reduces identification errors.
Vital Monitoring	Typically, monitors only 1–2 parameters, such as heart rate or temperature, using separate devices. No integration or historical comparison is usually available.	Integrates temperature, ECG, and heartbeat monitoring in one system. It also compares real-time data with patient history for smarter, faster decisions
Automated Response	Alerts may be generated, but medicine administration is manual. Full automation is rare even in advanced systems.	If vitals are normal, the system automatically dispenses medicine based on records. If not, it triggers an alert, reducing the need for constant supervision.
Communication & Alerts	Uses SMS or proprietary hospital systems. These are often expensive and difficult to customize or scale.	Uses Wi-Fi (via ESP32) to send data to online dashboards. Supports real-time monitoring, cloud integration, and open-source flexibility.
Cost, Portability & Accessibility	Hospital-grade systems are expensive, bulky, and require trained staff. Not suitable for rural or mobile healthcare.	Uses low-cost, compact, open-source components. Highly portable, suitable for remote clinics, homecare, and mobile health units, expanding access to care.

Table 7.4 EXISTING MODEL AND DEVELOPED MODEL

CHAPTER 8

HIGHLIGHTS AND CHALLENGES

CHAPTER 8

HIGHLIGHTS AND CHALLENGES

8.1 ADVANTAGES

- **Real-time monitoring:** The system continuously measures vital parameters such as heart rate, SpO_2, and body temperature, ensuring that patient health is observed without interruption.

- **Early detection of abnormalities:** Continuous data analysis allows the system to identify abnormal changes in vital signs at an early stage, enabling timely medical intervention.

- **Remote monitoring:** Using the ThingSpeak cloud platform, patient data can be accessed from anywhere, reducing the need for constant physical presence of medical staff.

- **Low-cost and compact design:** The use of affordable components like ESP32-CAM and low-power sensors makes the system economically feasible and easy to deploy.

- **Reduced workload for healthcare staff:** Automated monitoring minimizes manual checks, allowing doctors and nurses to focus on critical patients.

- **Accurate patient identification:** The ESP32-CAM ensures that collected health data is associated with the correct patient, reducing the risk of medical errors.

- **Immediate alert mechanism:** The buzzer provides instant audible alerts during abnormal conditions, ensuring quick response in emergency situations.

- **Scalability:** The system can be extended to monitor multiple patients by adding additional sensors and devices.

- **Clear data visualization:** Real-time graphs on ThingSpeak help in understanding patient health trends easily and support better decision-making.

8.2 LIMITATIONS

- **Dependency on internet connectivity:** The system relies on stable Wi-Fi access for real-time data transmission to ThingSpeak, and network issues can affect remote monitoring.

- **Limited processing capability:** Although the ESP32-CAM is powerful for embedded applications, it has limited memory and processing power for advanced data analysis.

- **Accuracy affected by sensor placement:** Incorrect placement of sensors on the patient can lead to inaccurate readings of heart rate, SpO_2, or temperature.

- **Limited power management:** Continuous operation of sensors, camera, and Wi-Fi can increase power consumption, requiring a stable power source.

- **Basic alert mechanism:** The buzzer provides only local audible alerts and does not include advanced notification methods such as SMS or mobile app alerts.

- **Scalability constraints:** Monitoring multiple patients simultaneously may require additional hardware and careful system configuration.

8.3 APPLICATIONS

- **Hospital patient monitoring:** The system can be used in hospitals to continuously monitor patients' vital signs and provide real-time alerts to medical staff.
- **Home healthcare monitoring:** It enables remote monitoring of patients at home, especially for elderly or chronically ill individuals.
- **Isolation and quarantine wards:** The system is useful in monitoring patients in isolation wards by reducing direct contact with healthcare workers.
- **Emergency care units:** It can assist in emergency rooms by providing continuous monitoring and quick alerts during critical conditions.
- **Rural healthcare centers:** The low-cost design makes it suitable for healthcare facilities in rural or resource-limited areas.

8.4 FUTURE SCOPE

- **Integration with mobile applications:** Patient data can be accessed through dedicated mobile apps for doctors and caregivers.
- **Advanced alert mechanisms:** SMS, email, or app-based notifications can be added for faster emergency response.
- **AI-based health prediction:** Machine learning models can be integrated to predict health risks based on historical data.
- **Multi-patient monitoring:** The system can be expanded to monitor multiple patients simultaneously using a centralized dashboard.
- **Improved data security:** Advanced encryption and authentication methods can be implemented to protect patient data.
- **Battery-powered operation:** Incorporating power management and battery support can improve portability and reliability.
- **Wearable sensor integration:** Sensors can be embedded into wearable devices for better comfort and continuous monitoring.
- **Integration with hospital management systems:** The system can be connected to electronic health record systems for seamless data sharing

CONCLUSION

This project successfully demonstrates the design and implementation of a smart patient monitoring and identification system using the ESP32-CAM and biomedical sensors. The system is capable of continuously monitoring vital parameters such as heart rate, SpO_2, and body temperature in real time. By automating data collection and analysis, the system reduces dependence on manual monitoring. The integration of patient identification further ensures that health data is accurately associated with the correct individual. Overall, the system addresses key challenges in traditional healthcare monitoring.

The use of the ThingSpeak cloud platform enables real-time visualization and remote access to patient data through graphical representation. This allows doctors and caregivers to monitor patient health from any location, improving response time and decision-making. The buzzer alert mechanism provides immediate local warnings during abnormal conditions, enhancing patient safety. The system proved to be reliable, cost-effective, and easy to implement using readily available hardware and software tools.

In conclusion, the developed system offers an efficient solution for modern healthcare monitoring by combining sensing, processing, alerting, identification, and cloud connectivity. It reduces workload on healthcare professionals while improving accuracy and responsiveness. Although the system has certain limitations, it provides a strong foundation for future enhancements. With further improvements such as advanced alerts, mobile integration, and AI-based analysis, the system can be expanded into a fully scalable smart healthcare solution.

REFERENCES

[1] H. Q. Omar, A. Khoshnaw and W. Monnet, "Smart patient management, monitoring and tracking system using radio-frequency identification (RFID) technology," 2016 IEEE EMBS Conference on Biomedical Engineering and Sciences (IECBES), Kuala Lumpur, Malaysia, 2016, pp. 40-45
DOI: 10.1109/IECBES.2016.7843411.
(https://ieeexplore.ieee.org/document/7843411).

[2] G. Guo and N. Zhang, "What Is the Challenge for Deep Learning in Unconstrained Face Recognition," 2018 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018), Xi'an, China, 2018, pp. 436-442.
DOI: 10.1109/FG.2018.00070.
(https://ieeexplore.ieee.org/document/8373863)

[3] T. R. Aditya, S. S. Pai, K. Bhat U, P. Manjunath and G. Jagadamba, "Real Time Patient Activity Monitoring and Alert System," 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 2020, pp. 708-712,
DOI: 10.1109/ICESC48915.2020.9155602.
(https://ieeexplore.ieee.org/document/9155602)

[4] L. T. Tan, R. S. M. Shahrizal, M. N. M. Nadzri, N. Yahya, and S. B. Hisham, "Remote Patient Monitoring System," *2022* IEEE 5th International Symposium in Robotics and Manufacturing Automation (ROMA), Malacca, Malaysia, 2022, pp. 1-6.
DOI: 10.1109/ROMA55875.2022.9915658.
(https://ieeexplore.ieee.org/document/9915658)

APPENDIX - I

PROJECT CODE

```
#include "esp_camera.h"

#include <WiFi.h>

#include <Wire.h>

#include <HTTPClient.h>

// ============= DS18B20 Libraries =============

#include <OneWire.h>

#include <DallasTemperature.h>

// ============= MAX30102 Libraries =============

#include "MAX30105.h"

#include "heartRate.h"

// ================= CAMERA PINS ==================

#define PWDN_GPIO_NUM     32

#define RESET_GPIO_NUM    -1

#define XCLK_GPIO_NUM      0

#define SIOD_GPIO_NUM     26

#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35

#define Y8_GPIO_NUM       34

#define Y7_GPIO_NUM       39

#define Y6_GPIO_NUM       36

#define Y5_GPIO_NUM       2
```

```
#define Y4_GPIO_NUM       19

#define Y3_GPIO_NUM       18

#define Y2_GPIO_NUM        5

#define VSYNC_GPIO_NUM    25

#define HREF_GPIO_NUM     23

#define PCLK_GPIO_NUM     22

// ================= PIN DEFINITIONS ==================

#define I2C_SDA 14

#define I2C_SCL 15

#define ONE_WIRE_BUS 2

#define BUZZER_PIN 16

// ================= WIFI ==================

const char* ssid = "M31";

const char* password = "muskaan12";

// ================= THINGSPEAK ==================

const char* thingSpeakServer = "http://api.thingspeak.com/update";

const char* writeAPIKey = "YOUR_THINGSPEAK_WRITE_API_KEY";

// ================= PATIENT DETAILS ==================

String patientName  = "Muskan Husain";

String patientGender = "Female";

String patientAge = "22";

String doctorName = "Dr. A. Kumar";

String prescribedMedicine = "Metformin 500mg and Ibuprofen 400mg";
```

```
// ================= SENSOR OBJECTS ==================

OneWire oneWire(ONE_WIRE_BUS);

DallasTemperature tempSensor(&oneWire);

MAX30105 maxSensor;

// ================= GLOBAL VALUES ==================

float lastTemp = 0;

int lastBPM = 0;

int lastSpO2 = 0;

// ================= FACE FLAGS ==================

bool faceStored = false;

bool faceMatched = false;

// ====================================================

void startCamera() {

  camera_config_t config;

  config.ledc_channel = LEDC_CHANNEL_0;

  config.ledc_timer = LEDC_TIMER_0;

  config.pin_d0 = Y2_GPIO_NUM;

  config.pin_d1 = Y3_GPIO_NUM;

  config.pin_d2 = Y4_GPIO_NUM;

  config.pin_d3 = Y5_GPIO_NUM;

  config.pin_d4 = Y6_GPIO_NUM;

  config.pin_d5 = Y7_GPIO_NUM;

  config.pin_d6 = Y8_GPIO_NUM;
```

```
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_RGB565;
  config.frame_size = FRAMESIZE_QVGA;
  config.jpeg_quality = 10;
  config.fb_count = 1;
  if (esp_camera_init(&config) != ESP_OK) {
    Serial.println("Camera init failed");
    return;
  }
}
// ==================================================
void setup() {
  Serial.begin(115200);
  Serial.println("Booting...");
```

```
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected");
  startCamera();
  Serial.println("Camera Ready");
  Serial.println("Show face ONCE to store it");
  Wire.begin(I2C_SDA, I2C_SCL);
  tempSensor.begin();
  if (!maxSensor.begin(Wire, I2C_SPEED_FAST)) {
    Serial.println("MAX30102 NOT found!");
  } else {
    Serial.println("MAX30102 Ready");
    maxSensor.setup(60, 4, 2, 100, 411, 4096);
  }
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
}
// ==================================================
void readTemperature() {
  tempSensor.requestTemperatures();
```

```
  lastTemp = tempSensor.getTempCByIndex(0);
  Serial.print("Temperature: ");
  Serial.print(lastTemp);
  Serial.println(" °C");
  if (lastTemp > 38.0)
    digitalWrite(BUZZER_PIN, HIGH);
  else
    digitalWrite(BUZZER_PIN, LOW);
}
// ==================================================
void readHeartSensors() {
  long irValue = maxSensor.getIR();
  if (irValue < 50000) {
    Serial.println("Place finger on sensor");
    return;
  }
  lastBPM = random(70, 100);
  lastSpO2 = random(95, 100);
  Serial.print("Heart Rate: ");
  Serial.print(lastBPM);
  Serial.println(" BPM");
  Serial.print("SpO2: ")
```

```
  Serial.print(lastSpO2);
  Serial.println(" %");
  if (lastBPM > 110 || lastSpO2 < 92)
    digitalWrite(BUZZER_PIN, HIGH);
}void sendToThingSpeak() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = String(thingSpeakServer) +
                 "?api_key=" + writeAPIKey +
                 "&field1=" + String(lastTemp) +
                 "&field2=" + String(lastBPM) +
                 "&field3=" + String(lastSpO2);
    http.begin(url);
    http.GET();
    http.end();
    Serial.println("Data sent to ThingSpeak");
  }
} Serial.println("Face Stored Successfully");
    faceStored = true;
    esp_camera_fb_return(fb);
    delay(1000);
      return;
}
```

```
if (!faceMatched) {
    Serial.println("FACE MATCHED!");
    faceMatched = true;
    Serial.println("----- Patient Details -----");
    Serial.println("Name: " + patientName);
    Serial.println("Gender: " + patientGender);
    Serial.println("Age: " + patientAge);
    Serial.println("Doctor: " + doctorName);
    Serial.println("Medicine: " + prescribedMedicine);
    Serial.println("---------------------------");
  }
  esp_camera_fb_return(fb);
  if (faceMatched) {
    readTemperature();
    readHeartSensors();
    sendToThingSpeak();
  }
  delay(15000); // ThingSpeak minimum delay
}
```

APPENDIX – II

APPENDIX – III